

■ ESTUDO FINANCEIRO E COMPORTAMENTAL DO BITCOIN

Pipeline ETL End-to-End

Trabalho Semestral · Data Integration · Sistemas de Informação · ESPM · 2026.1

Prof. Me. Andre Insardi

Nome	RA
Arthur Borba Ferreira	245559
Enzo De Marchi Malagoli	243089
Luis Felipe Galina Degaspari	238000
Luiz Felipe Pimenta Berrettini	245388
Luiz Fernando Pazdziora Costa	228615

1. Objetivo e Pergunta de Negócio

O Bitcoin é notoriamente volátil e fortemente influenciado pela psicologia de mercado. Este projeto integra duas fontes heterogêneas de dados para investigar a relação entre o **sentimento coletivo dos investidores** e a **variação de preço do ativo**, respondendo a perguntas relevantes para stakeholders de investimentos, risco e comunicação financeira.

Perguntas de negócio respondidas pelo pipeline:

- Quando o mercado está **ganancioso**, o Bitcoin de fato sobe mais — ou é aí que ele costuma corrigir?
- O **medo extremo** vem acompanhado de picos de volume (pânico de venda)?
- Como o **humor predominante** se distribuiu ano a ano entre 2018 e 2023?

2. Fontes de Dados

O projeto utiliza duas fontes intencionalmente heterogêneas — uma API REST estruturada e um CSV semi-estruturado —, atendendo ao requisito de diversidade de fontes do briefing.

#	Fonte	Tipo	O que extraímos
1	API REST Binance (/api/v3/klines)	API REST (estruturada)	OHLCV diário (abertura, fechamento, máxima, mínima, volume em USD). Market cap estimado: close × supply.
2	Kaggle — Bitcoin and Fear and Greed	CSV (semi-estruturado / sujo)	Índice Fear & Greed (0–100) e classificação (Extreme Fear ... Extreme Greed). Período: 2018–2023.

O CSV do Kaggle entra **crú** na camada raw — todas as colunas como texto, com nulos presentes — e é a fonte semi-estruturada exigida pelo briefing. A camada de transformação faz a limpeza, evidenciando os problemas de qualidade do dado bruto (3 linhas com sentimento nulo são corretamente descartadas).

3. Decisão de Engenharia: Troca da Fonte de Mercado

Esta seção documenta uma decisão técnica relevante tomada durante o projeto, com diagnóstico, justificativa e ressalvas.

3.1 O problema identificado

Na versão inicial, a **API da CoinGecko** era usada como fonte de preço. Após a execução do pipeline, as três consultas de valor retornavam vazias: o join entre preço e sentimento não casava nenhuma linha.

Fonte	Cobertura disponível (grátis)
CSV Kaggle Fear & Greed	2018-02-01 a 2023-03-31 (1.885 dias)

CoinGecko (plano gratuito)

Apenas os últimos 365 dias (2025–2026)

A documentação oficial da CoinGecko confirma que o plano público limita o histórico diário aos últimos 365 dias; acessar 2018–2023 exigiria plano pago. Como o CSV de sentimento termina em 2023, **não havia um único dia em comum** — daí o resultado vazio.

3.2 Solução adotada: Binance klines

A Binance expõe candles diários (OHLCV) via endpoint `/api/v3/klines` desde 2017, de forma **pública e sem chave de API**. Isso permitiu buscar exatamente a janela do CSV (parametrizada em `MARKET_START_DATE` / `MARKET_END_DATE`) e garantir a interseção necessária para o cruzamento.

3.3 O que não mudou (decisão consciente)

Para não alterar a lógica já validada, mantivemos os identificadores internos: a task continua `extract_coingecko`, a tabela `raw.coingecko_market` e o formato do payload são os mesmos. Apenas o *conteúdo* da extração passou a vir da Binance. Assim, transform, load, schema e as 3 consultas seguem idênticos.

3.4 Ressalvas da nova fonte

- **Market cap:** a Binance não fornece market cap. Ele é estimado como `close × supply_circulante` (~19,8 M BTC, parametrizável). O supply real cresce lentamente no tempo, gerando viés marginal no histórico — irrelevante para as análises comportamentais deste projeto.
- **Volume:** utilizado o *quote volume* em USDT ($\approx$ USD), par BTCUSDT — padrão de mercado, mas distinto de um preço médio global multi-exchange.

4. Arquitetura do Pipeline

O pipeline é orquestrado pelo **Apache Airflow** e persiste dados no **PostgreSQL** em três camadas no estilo *medallion*: `raw` → `staging` → `mart`.

4.1 Fluxo de tasks (DAG `btc_etl_pipeline`)

Task	Tipo	O que faz
<code>bootstrap_schema</code>	PythonOperator	Cria schemas/tabelas (idempotente) e popula <code>dim_sentiment</code>
<code>extract_coingecko</code>	PythonOperator	Chama API Binance → grava JSONB cru em <code>raw.coingecko_market</code>
<code>extract_kaggle_fng</code>	PythonOperator	Lê CSV Kaggle → grava texto cru em <code>raw.fear_greed</code>
<code>transform_market</code>	PythonOperator	<code>raw.coingecko_market</code> → <code>staging.btc_market</code> (tipado + DQ)
<code>transform_sentiment</code>	PythonOperator	<code>raw.fear_greed</code> → <code>staging.btc_sentiment</code> (limpo/normalizado)
<code>validate_staging</code>	PythonOperator	5 checagens DQ bloqueantes (fail-fast)
<code>load_dim_date</code>	PythonOperator	Popula <code>mart.dim_date</code> com calendário da janela
<code>load_fact</code>	PythonOperator	UPSERT em <code>mart.fact_btc_daily</code> (star schema)

validate_mart	PythonOperator	2 verificações de integridade referencial
---------------	----------------	---

4.2 Modelagem Dimensional (Star Schema)

Grão da tabela fato: **1 dia de mercado**.

Tabela	Tipo	Atributos principais
mart.fact_btc_daily	Fato	open/high/low/close_usd, volume_usd, market_cap_usd, daily_return_pct, intraday_range_pct, fng_value
mart.dim_date	Dimensão	date_key, full_date, year, quarter, month, day, is_weekend
mart.dim_sentiment	Dimensão	sentiment_key, classification, sentiment_order, sentiment_group
mart.dq_results	Observabilidade	check_name, status, linhas afetadas, msg, ts

5. Stack Tecnológica

Camada	Ferramenta	Versão
Linguagem	Python	3.11
Containerização	Docker + Compose	24+
Orquestração	Apache Airflow	2.9.3
Banco de Dados	PostgreSQL	15
ETL / manipulação	pandas	2.1.4
Conectividade BD	psycopg2 / SQLAlchemy	2.9.x / 1.4.52
Requisições HTTP	requests + tenacity (retry)	—
Logging	loguru	—
Testes	pytest	—

Observação: as versões de *pandas* (2.1.4) e *SQLAlchemy* (1.4.52) foram deliberadamente alinhadas às *constraints* oficiais do *Airflow* 2.9.3. Versões mais novas (ex.: *pandas* 2.2.x + *SQLAlchemy* 1.4) causam `AttributeError: 'Engine' has no attribute 'cursor'` no `pd.read_sql`, conforme diagnosticado e corrigido durante o desenvolvimento.

6. Validações de Qualidade de Dados

O módulo `etl/quality.py` implementa **5 checagens bloqueantes** em staging (a DAG para se alguma falhar — *fail-fast*) e **2 de integridade** pós-carga. Todos os resultados ficam registrados em `mart.dq_results` (observabilidade completa).

#	Checagem	Categoria	Tabela
---	----------	-----------	--------

1	not_null_close	Nulos	staging.btc_market
2	not_null_fng_value	Nulos	staging.btc_sentiment
3	unique_market_date	Duplicidade	staging.btc_market
4	range_fng_0_100	Range	staging.btc_sentiment
5	price_sanity (high >= low, valores > 0)	Range / consistência	staging.btc_market
6	fk_fact_dim_date	Integridade referencial	mart.fact_btc_daily
7	fk_fact_dim_sentiment	Integridade referencial	mart.fact_btc_daily

No dataset do Kaggle há 3 linhas com sentimento nulo que são corretamente descartadas na transformação — exatamente o tipo de problema que a camada de DQ existe para tratar.

7. As 3 Consultas de Valor

As consultas estão em `sql/analytics_queries.sql` e respondem diretamente às perguntas de negócio.

Consulta 1 — Retorno e volatilidade por sentimento

Pergunta: O humor do mercado se traduz em retorno? Quando os investidores estão gananciosos, o Bitcoin de fato sobe mais?

```
SELECT
    ds.classification                AS sentimento,
    ds.sentiment_order              AS ordem,
    COUNT(*)                        AS dias,
    ROUND(AVG(f.daily_return_pct), 4) AS retorno_medio_pct,
    ROUND(STDDEV_POP(f.daily_return_pct), 4) AS volatilidade_pct,
    ROUND(AVG(f.intraday_range_pct), 4) AS amplitude_media_pct
FROM mart.fact_btc_daily f
JOIN mart.dim_sentiment ds ON f.sentiment_key = ds.sentiment_key
GROUP BY ds.classification, ds.sentiment_order
ORDER BY ds.sentiment_order;
```

Consulta 2 — Volume e market cap por faixa do Fear & Greed

Pergunta: Existe relação entre o nível do índice e o volume negociado? O medo extremo gera pânico de vendas (volume alto)?

```
SELECT
    CASE
        WHEN f.fng_value < 25 THEN '0-24 (Medo Extremo)'
        WHEN f.fng_value < 45 THEN '25-44 (Medo)'
        WHEN f.fng_value < 55 THEN '45-54 (Neutro)'
        WHEN f.fng_value < 75 THEN '55-74 (Ganancia)'
        ELSE '75-100 (Ganancia Extrema)'
    END                               AS faixa_fng,
    COUNT(*)                          AS dias,
    ROUND(AVG(f.volume_usd) / 1e9, 2) AS volume_medio_bi_usd,
    ROUND(AVG(f.close_usd), 2)        AS preco_medio_usd,
    ROUND(AVG(f.market_cap_usd) / 1e9, 2) AS market_cap_medio_bi_usd
FROM mart.fact_btc_daily f
WHERE f.fng_value IS NOT NULL
GROUP BY 1
ORDER BY 1;
```

Consulta 3 — Resumo anual de desempenho e sentimento

Pergunta: Olhando por ano, qual foi o desempenho do Bitcoin e em que humor o mercado passou a maior parte do tempo?

```
SELECT
    dd.year                AS ano,
    COUNT(*)               AS dias_negociados,
    ROUND(MIN(f.low_usd), 2) AS minima_usd,
```

```

        ROUND(MAX(f.high_usd), 2)                AS maxima_usd,
        ROUND(AVG(f.close_usd), 2)              AS preco_medio_usd,
        ROUND(AVG(f.fng_value), 1)              AS fng_medio,
        MODE() WITHIN GROUP (ORDER BY ds.classification) AS sentimento_predominante
FROM mart.fact_btc_daily f
JOIN mart.dim_date dd      ON f.date_key = dd.date_key
LEFT JOIN mart.dim_sentiment ds ON f.sentiment_key = ds.sentiment_key
GROUP BY dd.year
ORDER BY dd.year;

```

Para executar as consultas com o ambiente rodando:

```
docker exec -i btc_postgres psql -U btc -d btc_dw < sql/analytics_queries.sql
```

8. Reprodutibilidade e Incrementalidade

- **Idempotência:** o bootstrap usa `CREATE TABLE IF NOT EXISTS`; a carga do fato usa `INSERT ... ON CONFLICT DO UPDATE (UPSERT)`, então reexecutar a DAG não duplica dados.
- **Do zero em outra máquina:** `docker compose up` sobe Postgres + Airflow já configurados; o CSV do Kaggle vai versionado em `data/seeds/`; a Binance é pública e não requer chave. Nenhum ajuste manual é necessário.
- **Janela parametrizável:** as datas de início/fim da extração de mercado são controladas pelas envs `MARKET_START_DATE` e `MARKET_END_DATE`, sem mudança de código.

9. Como Executar

1. Clone e configure

```

git clone <URL>
cd btc_pipeline
cp .env.example .env

```

2. Suba o ambiente

```
docker compose up -d --build
```

3. Acesse o Airflow

```
http://localhost:8080    •  admin / admin
```

4. Rode o pipeline

Ative a DAG `btc_etl_pipeline` e clique em Trigger DAG
ou: `make run-dag`

5. Veja os resultados

```

docker exec -i btc_postgres psql -U btc -d btc_dw
\i /opt/airflow/sql/analytics_queries.sql

```

6. Zere o ambiente

```
docker compose down -v (o -v apaga volumes/metadados)
docker compose build --no-cache airflow
docker compose up
```

10. Declaração de Uso de Inteligência Artificial

Conforme a seção 10 do briefing, declaramos o uso de IA neste trabalho:

- Usamos o **Claude (Anthropic)** para estruturar o esqueleto do pipeline, diagnosticar e corrigir conflitos de dependências (SQLAlchemy × pandas × Airflow), revisar a arquitetura em camadas e redigir a documentação técnica.
- Usamos o **GitHub Copilot** para acelerar a escrita de SQL, dos testes e de correções incrementais no código.
- Todo o código foi **revisado e compreendido** pelo grupo; somos responsáveis pelo que está entregue.

11. Referências

- BINANCE. *Kline/Candlestick Data — Spot API Documentation*. Disponível em: <https://developers.binance.com/docs/binance-spot-api-docs>.
- COINGECKO. *API Documentation* (fonte original, substituída). Disponível em: <https://docs.coingecko.com/>.
- BHATTI, A. *Bitcoin and Fear and Greed* [dataset]. Kaggle. Disponível em: <https://www.kaggle.com/datasets/adilbhatti/bitcoin-and-fear-and-greed>.
- APACHE AIRFLOW. *Documentation*. Disponível em: <https://airflow.apache.org/docs/>.
- KIMBALL, R.; ROSS, M. *The Data Warehouse Toolkit*. 3. ed. Wiley, 2013.